Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the Software Development Life Cycle

Spyridon Alvanakis Apostolou[1], Jan Bosch[1,2], and Helena Holmström Olsson[3]

[1] Chalmers University of Technology, Department of Computer Science and Engineering, Götenborg, Sweden spyalv@chalmers.se
[2] Eindhoven University of Technology, Department of Mathematics and Computer Science, Eindhoven, Netherlands jan.bosch@chalmers.se
[3] Malmö University, Department of Computer Science and Media Technology, Malmö, Sweden helena.holmstrom.olsson@mau.se

Abstract. Agentic AI in software product development is increasingly adopted by organizations, yet the field lacks a consolidated synthesis of where adoption is mature, which architectural patterns dominate, and what limitations and coping mechanisms exist in industrial deployments. This systematic literature review addresses these gaps by establishing a body of knowledge as a starting point. Following Kitchenham guidelines, we queried four major research databases, obtaining over 1600 candidate publications. To handle this volume, we developed and validated a domain-agnostic multi-agent screening pipeline that extends prior LLM-assisted review tools by combining automatic metadata curation, inter-agent iterative dialogue, and conflict-resolution defaults that minimize false negatives. From the 92 manually verified primary studies, our thematic synthesis reveals that output verifiability is the primary enabler of agentic adoption: later SDLC phases, whose outputs are objectively evaluable through executable feedback, demonstrate the highest maturity and industrial presence, while earlier phases remain almost exclusively academic proofs-of-concept. We identify the Planner–Executor–Reviewer role specialization as the dominant architectural pattern, with the Reviewer agent implementing verifiability through executable feedback loops. Across all challenge categories, industrial mitigation strategies converge on confining agent actions to verifiable, bounded spaces. This study contributes a comprehensive characterization of the current literature on agentic systems in software product development, and a methodological contribution in the form of an AI-assisted tool to automate the screening phase in high-volume SLR domains.

Keywords: Agentic AI, Autonomous Systems, LLM agents, Software Development Life Cycle, Software Product Development, Software Engineering, Systematic Literature Review

1 Introduction

The landscape of Artificial Intelligence has evolved rapidly with the advent of Large Language Models (LLMs). The era of generative AI (GenAI) was defined by "passive" interactions, where models generate text in response to explicit human prompts. According to Wang et al. [31], from 2023 the field has been progressing towards agentic AI, which, unlike standard reactive LLMs, is characterized by systems that demonstrate capabilities such as planning, reasoning, use of external tools, self-refinement, and execution of multi-step workflows while requiring minimal human intervention. With this task-oriented behavior, agentic AI pushes the boundaries of traditional LLMs across a wide variety of domains [1, 27].

This shift is particularly significant for Software Engineering (SWE). GenAI systems have been integrated into the Software Development Life Cycle (SDLC) as supportive assistants for tasks such as code completion, automated testing, and code generation. The emergence of agentic capabilities reframes the role of GenAI in SWE from assistance to autonomy [11]. Research is increasing rapidly, yet industrial adoption remains in its early stages: inherent LLM limitations, trust concerns, the lack of consolidated architectural patterns, and the rapid obsolescence of underlying models [25] stand as significant barriers to the integration of agentic systems into SDLC phases. As the volume of publications increases rapidly, a systematic synthesis of the current state-of-the-art is necessary. Consequently, this study conducts a Systematic Literature Review (SLR) to analyze and categorize the current state of agentic AI adoption and applications in the SDLC. This is further detailed in the following research questions (RQs):

1. In which phases of the SDLC is agentic AI currently demonstrating the highest maturity and industrial adoption?
2. What architectural patterns (e.g., multi-agent systems, tool use) are predominant in agentic AI implementations?
3. What are the reported challenges in using agentic AI in industrial environments, and how have these limitations been addressed?

The body of literature related to this domain is large and growing rapidly, making traditional manual filtering of publications extremely time-consuming. To address this, we developed and employed a multi-agent system using role-specific LLM agents (Assistant and Evaluator) to automate the screening process and efficiently identify the set of relevant publications.

The contribution of our study is two-fold: first, and most importantly, the results of the systematic literature review, which characterize the current state of agentic AI across the SDLC, while revealing output verifiability as the cross-cutting principle that connects phase maturity, dominant architectural patterns, and industrial mitigation strategies. Second, the multi-agent screening pipeline that extends prior LLM-assisted review tools with automatic metadata curation, inter-agent argumentative dialogue, and inclusion-biased conflict resolution, validated to achieve a low false-negative rate. The remainder of this paper is organized as follows: Section 2 analyzes the background of the concept and the

related work, Section 3 describes the methodology, Sections 4 and 5 present the research findings, answers to the research questions, and threats to validity, Section 6 presents the conclusions and future work, and Section 7 provides data and code availability.

2 Background

2.1 The Agentic Turn in SDLC

GenAI was integrated into SDLC phases as an assistant tool through reactive code completion and generation. Tools such as GitHub Copilot, Amazon CodeWhisperer [33], and Meta's CodeCompose [20] demonstrated measurable gains in developer productivity and code generation [5, 23]. Nevertheless, the assistants are bounded, responding to explicit prompts without taking autonomous actions beyond immediate output implementations.

As a part of progress, LLM agents extended this baseline by introducing tool integration and a sense-plan-act structure, enabling task-specific automation. However, individual agents remain narrow in scope and operate independently across tasks, lacking features that enhance autonomy [27]. Agentic AI goes further, with capabilities such as task decomposition, persistent memory, and orchestrated autonomy, these systems pursue complex, multi-step goals with minimal human intervention [1, 27, 28]. Practically, this distinction means agentic systems can span entire SDLC phases rather than individual developer actions.

2.2 Related Work

Research activity in this field has grown substantially since 2023, when the first agentic frameworks for end-to-end development and role-specialized multi-agent pipelines began to emerge [31]. Several recent secondary studies approach the topic from complementary angles. He et al. [10] examined 71 primary studies on LLM-based multi-agent systems across the SDLC. Their review is paired with case studies that surface capability limits, and it concludes with a two-phase research agenda focused on individual agent enhancement and, subsequently, agent synergy. A broader synthesis is offered by Liu et al. [18], who survey 106 papers through a dual lens: an SWE perspective spanning individual tasks and end-to-end workflows, and an agent-centric view that analyzes planning, memory, perception, action, and collaboration. Wang et al. [32] take a different direction. Working through 115 papers under a perception-memory-action framework, they flag open issues such as under-explored modalities, hallucination mitigation, and inefficiencies in multi-agent coordination. Otoum and Elkhalili [21] narrow the lens to method categories, namely autonomous coding, multi-agent systems, iterative refinement, and human-agent collaboration, classifying 61 studies along these axes while tracing their temporal evolution and pointing to an early-phase autonomy gap.

Looking beyond software-specific reviews, Bandi et al. [4] cover broader ground. Their analysis of 143 studies across multiple domains contributes concept-level

disambiguation of agentic AI paradigms, a comparison of major LLM frameworks, and five architectural models accompanied by evaluation-metric taxonomies. Hosseini and Seilani [11] shift the discussion toward organizational strategy, proposing an adoption framework that addresses business needs, tool selection, and risk management. The practitioner viewpoint is captured by Akbar et al. [3]: drawing on interviews with 21 experts, they argue that agentic AI redefines traditional SDLC phase boundaries through continuous feedback loops and real-time optimization.

We systematically examine peer-reviewed publications on agentic AI across the SDLC, contrasting academic proof-of-concept (PoC) studies with industrially adopted implementations to resolve SDLC-phase maturity (RQ1), architectural patterns in relation to agent autonomy (RQ2), and reported challenges paired with their mitigation strategies in industrial context (RQ3). Critically, rather than organizing findings around agent capabilities or method categories in isolation, our synthesis surfaces output verifiability as the cross-cutting principle that connects phase distribution, architectural design, and industrial mitigation strategies. Beyond the SLR results, the study makes a methodological contribution of a domain-agnostic multi-agent screening pipeline that extends existing LLM-assisted review tools with automatic metadata curation, inter-agent argumentative dialogue, and inclusion-biased conflict resolution, applicable to high-volume SLRs beyond this domain.

3 Methods

3.1 Research Design

As the basis for this SLR, we followed the Kitchenham and Charters guidelines [16]. We consulted an initial list of databases that included: Google Scholar, Scopus, IEEE Xplore, ACM Digital Library, and ScienceDirect. That initial list was further refined to address reproducibility and search functionality issues. Google Scholar was excluded because of the time and location dependence of its results, in addition to an insufficient character limit [8]. ScienceDirect was excluded due to its very restrictive term and boolean limit in advanced search [8]. The refined list, using the terminology defined in [8], consisted of the following libraries:

- IEEE Digital Library http://ieeexplore.ieee.org
- ACM Digital Library http://dl.acm.org
- SpringerLink https://link.springer.com/
- Scopus https://www.scopus.com

3.2 Query

In order to translate the study goal with the research questions to a query structure, we utilized the CIMO (Context, Intervention, Mechanism, Outcome) logic [6]. The key data points extracted included:

- Context: The specific problem domain or application area.
- Intervention: The techniques and methods used at the context.
- Mechanism: The underlying methods and tools employed.
- Outcome: The results achieved, including performance improvements.

The implemented query is the following:

```
("software lifecycle" OR "software development" OR "software product")
AND
("agentic" OR "autonomous agent*" OR "goal-driven agent*" OR "LLM agent*"
     OR ("multi-agent" AND"LLM"))
AND
("tool use" OR "self-reflection" OR "task decomposition" OR "
    orchestration" OR "proactive" OR "goal-driven" OR "task oriented" OR
    "self-adapted")
AND
("automation" OR "accelerat*" OR "efficiency")
```

3.3 Time frame, Inclusion & exclusion criteria

Since the study focuses on agentic AI in SPD, we limited our review to publications from 2023 onwards, aligning with the timeline identified by Wang et al. [31] who identifies the first agentic frameworks to develop, and thus initiate the transition from standard LLMs to agentic AI as a distinct phase beginning with the advanced reasoning capabilities of models released in early 2023. To avoid collecting unnecessary publications, we employed the timeframe criteria as a filter at the advanced search phase along with the query implementation to each database. Additionally, in order to ensure adherence to the review scope, the inclusion and exclusion criteria shown in Table 1 were curated to be applied to the initial search results. These criteria were designed using the guidance in Kitchenham and Charters [16] and Gough et al. [7].

Along with the time frame filter, we implemented the filter of language during the raw data collection phase when filtered data resulted from the query into the diverse databases.

3.4 Data analysis method

Agentic AI is a rapidly evolving domain in which the literature grows continuously, making manual review at scale impractical. This motivated the development of an agile LLM-based tool to reduce reviewer workload. Existing tools such as Rayyan [22] and Abstrackr [30] rely on active learning to prioritize relevant studies. More recently, LatteReview [26] employs role-playing LLM agents with hierarchical screening and reasoning transparency. However, LatteReview requires users to manually script agent personas and scoring rules, and expects a complete dataset as input.

To further automate the process and reduce both prompt subjectivity and false negatives, we developed a model- and domain-agnostic six-step pipeline. It requires

Table 1. Inclusion Criteria

ID	Criteria
1	Publication date within the defined time frame.
2	A primary study, not a secondary or a tertiary study.
3	Published in a peer-reviewed journal, conference proceeding, or book chapter not in grey literature reports, blog posts, or non-academic publications.
4	Published in English.
5	Not a duplicate or a version of another included study.
6	Discusses LLM agents with particulars matching the agentic characteristic.
7	Focuses in software product development.
8	Study provides a Proof of Concept (not a vision study).

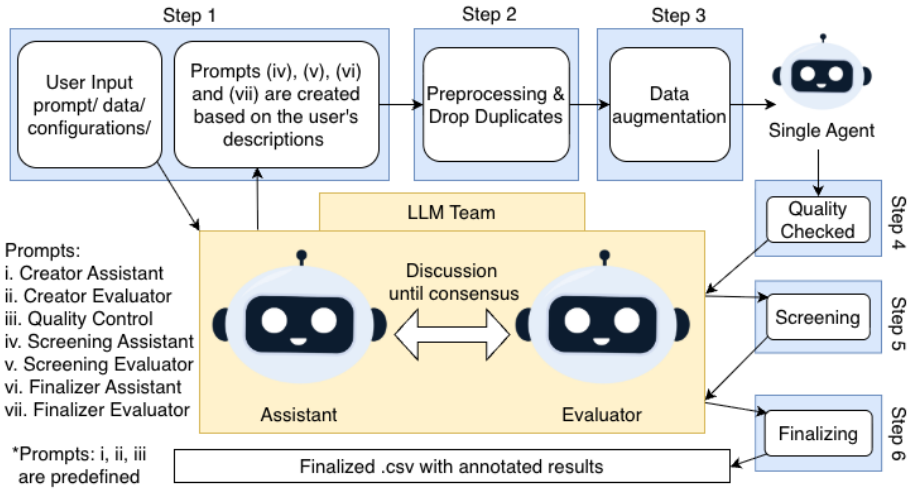

Fig. 1. Analytical workflow of the multi-agent collaboration and consensus mechanism.

minimal reviewer input, raw publication metadata (title, DOI, abstract, etc.) and a single detailed prompt containing the research purpose, research questions, and selection criteria. The pipeline includes self-curation of missing abstracts, inter-agent classification discussions, and produces binary relevant/irrelevant labels with full transparency of decisions, reasoning, and agent dialogue.

In the first step of the workflow, two role-playing agents (Assistant and Evaluator) based on the user-provided prompt, respectively they generate and evaluate task-specific prompts for the subsequent phases, only after the reviewer approves the final prompt of each task the process proceeds to the next step. In the second step, raw exports from multiple databases (CSV, RIS, or BIB format)

are normalized into an unified CSV with duplicates records removed. In the third step, a self-curation process retrieves missing abstracts via web scraping and API-driven augmentation.

From step four onward, records are filtered against the inclusion criteria. In the Quality Control step, a single agent excludes non-research items (e.g., conference preambles, generic book chapters). A human verification of excluded records lacking abstracts is recommended at this stage to prevent accidental omission of legitimate studies.

In the Screening step, the Assistant and Evaluator agents independently classify each publication against the inclusion/exclusion criteria and high-level domain relevance. It is important to note here, that for every double-agent step, we employed different models in order to minimize classification errors when independent reasoning paths reach the same conclusion. In case of conflict, a structured inter-agent dialogue with iterative argumentation (up to three rounds or until agreement) is triggered to finalize the classification label, unresolved conflicts default to inclusion to minimize false negatives. The purpose of this dialogue is to leverage one agent's reasoning to challenge the other's classification, serving as an additional de-hallucination mechanism. The final Relevance Selection step applies the same multi-agent process, now focused on alignment with the research questions and scope. The complete workflow is illustrated in Figure 1.

The two-phase (screening and relevance) design serves three purposes: (1) modularity, allowing costlier models to be selectively applied to the smaller candidate set in the final phase only, (2) reduced agent cognitive load, as each phase operates under a constrained set of criteria, and (3) independent evaluability of each stage.

Following the automated pipeline, the candidate relevant publications underwent manual evaluation following a structured two-pass protocol. In the first pass, each selected as relevant study's abstract was first read and applied Criteria 6, 7, and 8 from Table 1 to confirm or reject the automated label. In the second pass, studies confirmed as relevant were read in full to extract responses to at least one of the research questions. Data extraction was guided by a pre-defined coding sheet capturing: (i) targeted SDLC phase, (ii) evaluation context (academic benchmark vs. industrial deployment), (iii) architectural pattern, and (iv) reported limitations and mitigations.

4 Results

4.1 Query and Data Analysis Implementation

The search was executed across the databases and search systems described in Section 3.1, using the query as described in Section 3.2. Table 2 summarizes the results acquired from each phase of the process.

We initially provided the prompt-creation agents with a formal definition and a detailed description of the agentic AI concept, together with all the research questions and the exclusion criteria (except the language and time-range filters,

which had already been applied at the database query stage). Based on these inputs, four prompts were automatically generated for the screening and relevance phases. We employed bigger models for the prompt-generation phase (Assistant (A): gpt-5.4, Evaluator (E): gpt-5.2) to ensure high-quality prompt generation, and smaller yet capable models for the subsequent phases to manage cost (A-scr: gpt-5-mini, E-scr: gpt-5-nano, A-rel: gpt-5-mini, E-rel: deepseek-v3).

Table 2. Search Process Results

Source	**Raw**	**Proces.**	**QC**	**SC**	**Relev.**	**Manual Eval.**
ACM	659	605	139	44	16	9
IEEE	289	284	247	106	55	41
Scopus	485	326	315	88	43	32
Springer	276	116	95	27	13	10
Total	**1609**	**1331**	**796**	**265**	**127**	**92**

As illustrated in Table 2, a significant number of records corresponded to generic conference introductions rather than actual research publications and were consequently excluded. After completing the quality-control step, the final number of valid publications was 796. A manual inspection of the generated annotations from this step revealed that only 3 out of the 1331 retrieved records were still missing abstracts while still representing legitimate research publications.

We applied the final two steps of the process. It is worth noting that the screening phase is the most time-consuming step, as a large number of quality checked publication's abstracts must be evaluated by two independent agents. Out of 796 valid publications, 265 passed the screening phase, the remaining 531 were excluded for failing to meet the inclusion criteria beyond those already enforced (date range, language, and publication type, Criteria 1, 3, and 4 in Table 1). The final relevance-selection step identified 127 candidate studies. Following the manual evaluation procedure described in Section 3.4, which involved reading the abstracts of all 127 papers, we identified 101 as conceptually relevant, from which we read the full text. Finally, 92 were confirmed as the final relevant set, since 9 papers lacked a PoC and were excluded. Consequently, we identified 26 as false positives produced by the automated pipeline, since only the abstracts were provided to the agents. In addition to this manual verification, a random sample drawn from the quality-control and screening exclusion sets was evaluated to further validate the selection process and ensure that false-negative classifications were minimized.

False negatives evaluation To evaluate the pipeline's reliability, we sampled 100 publications that were excluded by the agent-based stages: 50 excluded during

the screening phase and 50 excluded during the final relevance selection phase. Following the central limit theorem rule of thumb, a minimum of 30 samples per stage was considered sufficient to approximate normality in the sampling distribution, and this is why we consider the samples of 50 sufficient. Manual evaluation of each sample was performed to identify any false negatives.

The single false negative identified across the full sample of 100 came from the relevance-selection phase (1/50), while no misclassifications were observed in the screening step (0/50). Extrapolating the observed rate across the full excluded population (669 papers), suggests approximately 7 missed relevant publications in total, and since the sole missed publication originated from the relevance-selection phase, which operates on a smaller candidate set, the true number is likely even lower.

4.2 Thematic synthesis

Combining the information from the final set of 92 publications, we structured the answers to our initial research questions.

RQ1 In which phases of the SDLC is agentic AI currently demonstrating the highest maturity and industrial adoption?

The phase distribution of the final publications set is summarized in Table 3. From 92 studies only 13 were developed and evaluated in an industrial context, while the rest 79 represent academic PoC studies evaluated on benchmarks or controlled experimental settings. For both types of studies we can observe from Table 3 that the phases of the SDLC that demonstrate higher representation are **Testing & QA**, **Maintenance**, **Deployment**, and **Coding & Implementation** indicating that agentic AI maturity is concentrated in post-development phases.

The dominance of post-development phases reflects an important characteristic. They include tasks from which the performance can be verified through objective, executable feedback (e.g., test results, compiler outputs, fault localization traces, and log signals). These concrete results provide the agents a specific state to improve during self-refinement loops, which can be implemented without human evaluation.

On the industrial side, the concentration in Testing & QA reflects the state, that testing is time consuming and labour intensive of the SDLC, and its outcomes are verifiable. Publications in this concept target high-cost bottlenecks with clear success criteria, from GUI testing of production desktop and web applications to automotive software release gatekeeping [15] and structural testing of agentic systems themselves [17]. Moreover, Deployment & Operations tasks follows the same logic, as AIOps agents monitor structured operational signals (e.g., metrics, logs, and CI/CD states), which outputs can be evaluated against concrete production thresholds for service performance anomaly detection [13, 15].

The underrepresentation of Coding & Implementation and Maintenance with industrial implementation does not necessarily translate as lack or research

effort in these areas. Academic studies classified under these phases demonstrate that the majority rely on test execution as primary verification mechanism. Studies targeting code generation, automated debugging, and program repair (corresponding examples of systems: EvoMAC, SoapFL, and RGD), validate their contributions through unit tests or benchmarks such as HumanEval or Defects4J [12, 14, 24]. Notably, Maintenance, Coding & Implementation from the literature are not fully distinguished by the Testing & QA phase, since the correctness of the agents outputs is eventually evaluated by a form of testing. This suggests that Testing & QA represents an entry point for industrial agentic adoption, while the integration of agentic systems in Maintenance or Coding & Implementation require agents to operate on large industrial codebases with legacy systems, undocumented constraints, and evolving requirements that controlled benchmarks do not capture.

Table 3. Distribution of Primary Studies by SDLC Phase and Evaluation Context

SDLC Phase	**Industrial**	**Academic**	**Total**
Maintenance	1	19	20
Testing & QA	5	13	18
Cross-cutting (multiple phases)	4	11	15
Deployment & Operations	2	12	14
Coding & Implementation	0	12	12
Requirements Analysis	0	7	7
Project Management	1	2	3
Design & Architecture	1	2	3
Total	**13**	**79**	**92**

Finally, Requirements Analysis, Design & Architecture, and Project Management represent the least explored phases, almost exclusively in academic settings. While limited in volume, the industrial context studies in Design & Architecture and Project Management demonstrate that meaningful agentic integration is feasible even in early SDLC stages, ranging from formal architectural verification [29] to agile sprint planning automation [2].

RQ2 What architectural patterns (e.g., multi-agent systems, tool use) are predominant in agentic AI implementations?

The predominant architecture across the selected studies is multi-agent role specialization. In this concept complex SWE tasks are decomposed in sequential or graph-based pipelines of agents, each assigned with a specific and bounded responsibility. The most common structure is Planner → Executor → Reviewer,

with an Orchestrator agent managing the flow of the sub-processes. Notably, this structure pattern appeared in all the SDLC phases publications. Moreover, the rationale is to reduce the cognitive load of the agents by narrowing their scope in order to produce more predictable and accurate outcomes, while additionally structured typed inter-agent communication methods (e.g., via JSON or LangGraph state graphs) in many occasions replaced free text handoffs with formally typed state transitions. A key example of an industrial case study, illustrates why this structure succeeds in practice. A two-agent Planner–Actor configuration is used for software release gatekeeping [15], where the Planner applies Chain-of-Thought reasoning with self-consistency to generate a release strategy, and the Actor executes from a predefined atomic action vocabulary with self-reflection for error correction. Restricting the Actor to a fixed action set was key to industrial adoption, as it bounds operational risk without sacrificing the reasoning flexibility of the Planner, achieving a separation between what to do and how to do it safely.

Beyond agent role decomposition, memory and retrieval architectures form the second structural axis of the surveyed systems. For memory and information retrieval with Retrieval-Augmented Generation (RAG) is implemented as baseline mechanism for injecting domain-relevant context. Interestingly, we mostly find variations in industrial studies. For an agentic RAG system (validated in production settings at Apple) [9], the authors developed a hybrid vector-graph knowledge system to preserve structural relationships between test artifacts and requirements, addressing this way the loss of relational context in flat retrieval. Similarly, the Codebase-Aware Generative Agents system [19] uses a Zero-Shot Dependency Mapping Knowledge Graph as a shared memory layer across agents. By combining deterministic static code parsing with constrained LLM extraction, this approach enables reasoning over structural code relationships, addressing architectural blindness.

RQ3 What are the reported challenges in using agentic AI in industrial environments, and how have these limitations been addressed?

This research question focuses on industrial studies, in which agentic systems are developed and evaluated under real-world company-specific contexts. Before we dive into it is important to note that there is a difference on perspective between academic and industrial based studies, nevertheless the limitations are similar. Academic implementations mostly focus on performance and feasibility under controlled experimental conditions, while industrial case studies attempt to address reliability issues to ensure consistency. Overall, the most common limitations we found in studies are the following: Non-deterministic behavior, information overload, hallucinations, and scalability.

Non-Determinism and Hallucinations: Non-determinism and hallucinations in LLM outputs is universally acknowledged. In most studies (from both categories) the predominant coping mechanism is iterative self-correction loops, in which systems identify their own mistakes bases on results from the evaluation method (e.g., testing, compiler output, logs, metrics). An extra feature to narrow

the scope of agents is role assignments with detailed instructions or chain of thought to provide information about steps, rather than letting the agents finding them on their own. Industrial studies, while adopting these coping mechanisms, implement additional constraint layers. More specifically, systems developed for domain-specific information implement a terminology knowledge base with semantic meanings to enhance the understanding of the agents, additionally in the same study, the complexity of the two-agent system were mostly assigned to one agent for transparency and interpretation reasons, while the second agent had a pre-defined action space [15]. The latter is widely employed in industrial studies with variations (tool calling or API communication) or structured outputs of agents in order to reduce hallucinations from unstructured text. Notably, in a different perspective industrial study instead of trying to force the AI to be perfectly predictable, built a rigorous, automated safety net around the AI to catch it if it makes a mistake [17], representing a shift that agents are not only tools but also subjects of verification.

Scalability: Agentic AI systems in industrial contexts mostly operate on large codebases, distributed microservices, and continuous streams of logs. More specifically, scalability concerns in industrial contexts span computational cost, execution time, and context size. Multi-agent iterative loops and tree-search strategies (e.g., Monte Carlo exploration) introduce costs that grow non-linearly with task complexity. While in most studies task-decomposition is enough narrow the scope of the agent, cases that manage real time monitoring needs extra steps. For example, systems employed for continuous and massive-scale operations utilize a combination of decentralized execution with task delegation in containerized Kubernetes environments, and dynamic stream processing via engines that compute metrics over sliding time windows of data [13]. Other common solutions, target stage-specific pass/fail tests to reject intermediate errors directly rather than propagate them to later pipeline steps.

Context Management and Information Overload: In industrial environments, the information stream is heterogeneous and continuous, as context is imported from source code, execution logs, documentation, and architectural artifacts. This limitation is not independent of the aforementioned ones, since in many occasions large information that exceeds the context window increases the likelihood of hallucinations or incomplete answers, while additionally requires more computational time increasing the latency and the demand of sources to avoid bottlenecks. As described in RQ2, a common mitigation strategy in both industrial and non-industrial studies is the use of RAG to address this challenge. However, specifically in industrial contexts, a flat vector retrieval alone is unable to preserve structural relationships between artifacts. The CA-SAF system [19] addresses this with a Zero-Shot Dependency Mapping Knowledge Graph, replacing flat retrieval with graph-based reasoning over structural code relationships. Similarly, another testing system uses a hybrid vector-graph representation that preserves business-logic relationships between test artifacts and requirements [9]. These approaches represent architectures where retrieval is structurally grounded rather than semantically approximate.

4.3 Synthesis and Discussion

Examining the three RQs jointly reveals a unifying principle: *output verifiability* operates as the primary enabler of agentic adoption, and its influence is traceable across phase distribution, architectural design, and industrial mitigation strategy.

RQ1 shows that the phases with the highest maturity and industrial presence (Testing & QA, Deployment & Operations, and Maintenance) are those whose outputs are objectively evaluable via executable feedback. Requirements Analysis and Design & Architecture, remain almost exclusively in academic PoC territory. This asymmetry suggests that the current ceiling of industrial agentic adoption is defined by the availability of ground-truth feedback at the task level.

RQ2 highlights this at the architectural level. The dominant Planner-Executor-Reviewer pattern is not only a coordination structure, the Reviewer agent is the verifiability mechanism, providing the feedback that grounds iterative refinement. The industrial preference for hybrid vector-graph retrieval over flat RAG adds a structural layer for verifiability to the information-access layer. Finally, RQ3 shows that industrial mitigations across all challenge categories converge on confining agent actions to bounded, verifiable spaces (predefined actions, structured outputs, stage-wise pass/fail tests).

These observations imply that extending agentic integration into early SDLC phases is, first and foremost, a problem of designing phase-appropriate feedback mechanisms rather than of improving model capability.

5 Threats to Validity

Threats to Internal Validity: The search strategy was systematically defined and implemented across multiple major databases, although relevant studies may have been omitted due to query keyword limitations or database coverage. The automated pipeline was designed to minimize false negatives, to validate this we sampled excluded publications from two distinct phases of the six-step process and projected the estimated false-negative rate ($\approx$1%) to the larger excluded set, estimating a $\approx$7 misclassified studies. Furthermore, relevance decisions depend on predefined prompts and specific LLM models, meaning prompt inaccuracies or future model changes may introduce systematic variability in classification outcomes that cannot be fully controlled.

Threats to External Validity: This review focuses on peer-reviewed publications from 2023 onwards. Grey literature was deliberately excluded, as peer-reviewed studies provide methodological transparency and independent validation, however this entails an information loss since industrial agentic AI practices frequently emerge in grey literature prior to formal publication. Additionally, many industrial studies rely on proprietary data and organization-specific workflows, limiting the transferability of reported architectures and mitigation strategies across contexts.

Threats to Temporal Validity: Given the rapid evolution of agentic AI, the findings reflect the literature available at the time of data collection. However,

the identification of open challenges and research gaps provides a forward-looking perspective that partially mitigates this limitation.

6 Conclusions And Future Work

This systematic literature review synthesizes the current state of agentic AI in software product development based on 92 primary studies identified through a structured search across four major scientific databases. The screening process was accelerated by a multi-agent pipeline that extends existing LLM-assisted review approaches with automatic metadata curation, inter-agent dialogue, and inclusion-biased conflict resolution, validated to achieve a low false negative rate.

The thematic synthesis across the three research questions reveals a unifying principle: output verifiability is the primary enabler of agentic AI adoption. RQ1 shows that the phases with the highest industrial presence (Testing & QA, Deployment & Operations, and Maintenance) are those whose outputs are objectively evaluable through executable feedback (test results, compiler outputs, operational metrics). Earlier lifecycle phases remain almost exclusively as purely academic PoCs, constrained by the absence of ground-truth feedback signals. RQ2 identifies the Planner-Executor-Reviewer role specialization as the dominant architecture, where the Reviewer implements verifiability within iterative refinement loops. RQ3 shows that across reported challenge categories (non-determinism, hallucinations, scalability, and context overload), industrial mitigation strategies consistently converge on confining agent actions to verifiable, bounded spaces.

Future Work: The most consequential open direction is extending agentic integration into other SDLC phases, which requires establishing phase-appropriate verifiability mechanisms to enable the iterative feedback loops that currently drive adoption in post-development stages. A second direction concerns the long-term assessment of production-scale agentic systems, evaluating their maintainability, adaptability, and reliability under evolving codebases and shifting requirements, conditions that controlled benchmarks do not capture. Finally, establishing empirical comparison protocols between multi-agent architectures across SDLC phases would allow practitioners to make principled decisions about agent autonomy and orchestration design.

7 Data and Code availability

To facilitate the reproducibility of our methodology, we have made the complete replication developed tool available in an online repository (click here). This repository contains the source code of the developed multi-agent pipeline, the specific prompts utilized for the Assistant and Evaluator agents, and the raw datasets resulting from the query implementation across the four queried databases.

References

1. Acharya, D.B., Kuppan, K., Divya, B.: Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey. IEEE Access **13**, 18912–18936 (2025), https://ieeexplore.ieee.org/abstract/document/10849561
2. Adapa, C., Anjana, A., Rahim, R., Victor, A.: A multi-agent ai framework for agile workflow automation, issue resolution, and developer performance evaluation. In: 2025 IEEE International Conference for Women in Innovation, Technology & Entrepreneurship (ICWITE). pp. 1–6. IEEE (2025)
3. Akbar, M.A., Khan, A.A., Hamza, M., et al.: Agentic AI in Software Engineering: Practitioner Perspectives Across the Software Development Life Cycle (Sep 2025), https://papers.ssrn.com/abstract=5520159
4. Bandi, A., Kongari, B., Naguru, R., et al.: The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges. Future Internet **17**(9) (Sep 2025), https://www.mdpi.com/1999-5903/17/9/404
5. Becker, J., Rush, N., Barnes, E., Rein, D.: Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (Jul 2025), http://arxiv.org/abs/2507.09089
6. Denyer, D., Tranfield, D., van Aken, J.E.: Developing Design Propositions through Research Synthesis. Organization Studies **29**(3), 393–413 (Mar 2008), https://doi.org/10.1177/0170840607088020
7. Gough, D., Thomas, J., Oliver, S.: An introduction to systematic reviews. SAGE (2017)
8. Gusenbauer, M., Haddaway, N.R.: Which academic search systems are suitable for systematic reviews or meta-analyses? Research Synthesis Methods **11**(2), 181–217 (2020), https://onlinelibrary.wiley.com/doi/abs/10.1002/jrsm.1378
9. Hariharan, M., Arvapalli, S., Barma, S., Sheela, E.: Agentic RAG for Software Testing with Hybrid Vector-Graph and Multi-Agent Orchestration (Oct 2025), http://arxiv.org/abs/2510.10824
10. He, J., Treude, C., Lo, D.: LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead (Jul 2025), http://arxiv.org/abs/2404.04834
11. Hosseini, S., Seilani, H.: The role of agentic AI in shaping a smart future: A systematic review. Array **26**, 100399 (Jul 2025), https://www.sciencedirect.com/science/article/pii/S2590005625000268
12. Hu, Y., Cai, Y., Du, Y., et al.: Self-Evolving Multi-Agent Collaboration Networks for Software Development (Oct 2024), http://arxiv.org/abs/2410.16946
13. Ji, X., Zhang, L., Zhang, W., et al.: LEMAD: LLM-Empowered Multi-Agent System for Anomaly Detection in Power Grid Services. Electronics **14**(15), 3008 (Jan 2025), https://www.mdpi.com/2079-9292/14/15/3008
14. Jin, H., Sun, Z., Chen, H.: RGD: Multi-LLM Based Agent Debugger via Refinement and Generation Guidance (Oct 2024), http://arxiv.org/abs/2410.01242
15. Khoee, A.G., Yu, Y., Feldt, R., et al.: GoNoGo: An Efficient LLM-based Multi-Agent System for Streamlining Automotive Software Release Decision-Making (Sep 2024), http://arxiv.org/abs/2408.09785
16. Kitchenham, B., Charters, S., et al.: Guidelines for performing systematic literature reviews in software engineering. Keele (2007)
17. Kohl, J., Kruse, O., Mostafa, Y., et al.: Automated structural testing of LLM-based agents: methods, framework, and case studies (Jan 2026), http://arxiv.org/abs/2601.18827

18. Liu, J., Wang, K., Chen, Y., et al.: Large Language Model-Based Agents for Software Engineering: A Survey (Dec 2025), http://arxiv.org/abs/2409.02977
19. Mavani, V.K.: Codebase Aware Generative Agents for the SDLC: Automating Documentation, Dependency Analysis and Test Generation. In: 2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC). pp. 1–4 (Feb 2026), https://ieeexplore.ieee.org/document/11395666
20. Murali, V., Maddila, C., Ahmad, I., et al.: AI-Assisted Code Authoring at Scale: Fine-Tuning, Deploying, and Mixed Methods Evaluation. Proc. ACM Softw. Eng. **1**(FSE), 48:1066–48:1085 (Jul 2024), https://dl.acm.org/doi/10.1145/3643774
21. Otoum, N., Elkhalili, N.: Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review. IEEE Access **14**, 7443–7465 (2026), https://ieeexplore.ieee.org/abstract/document/11343819
22. Ouzzani, M., Hammady, H., Fedorowicz, Z., Elmagarmid, A.: Rayyan—a web and mobile app for systematic reviews. Systematic Reviews **5**(1), 210 (Dec 2016), https://doi.org/10.1186/s13643-016-0384-4
23. Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M.: The Impact of AI on Developer Productivity: Evidence from GitHub Copilot (Feb 2023), http://arxiv.org/abs/2302.06590
24. Qin, Y., Wang, S., Lou, Y., et al.: SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization. IEEE Transactions on Software Engineering **51**(4), 1173–1187 (Apr 2025), https://ieeexplore.ieee.org/document/10891926
25. Raheem, T., Hossain, G.: Agentic AI Systems: Opportunities, Challenges, and Trustworthiness. In: 2025 IEEE International Conference on Electro Information Technology (eIT). pp. 618–624 (May 2025), https://ieeexplore.ieee.org/abstract/document/11103638
26. Rouzrokh, P., Khosravi, B., Rouzrokh, P., Shariatnia, M.: LatteReview: A Multi-Agent Framework for Systematic Review Automation Using Large Language Models (Oct 2025), http://arxiv.org/abs/2501.05468
27. Sapkota, R., Roumeliotis, K.I., Karkee, M.: AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges. Information Fusion **126**, 103599 (Feb 2026), http://arxiv.org/abs/2505.10468
28. Schneider, J.: Generative to Agentic AI: Survey, Conceptualization, and Challenges (Apr 2025), http://arxiv.org/abs/2504.18875
29. Tahat, A., Amundson, I., Hardin, D., Cofer, D.: Agree-dog copilot: a neuro-symbolic approach to enhanced model-based systems engineering. In: International Conference on Bridging the Gap between AI and Reality. pp. 117–137. Springer (2025)
30. Wallace, B.C., Small, K., Brodley, C.E., et al.: Deploying an interactive machine learning system in an evidence-based practice center: abstrackr. In: Proceedings of the 2nd ACM SIGHIT International Health Informatics Symposium. pp. 819–824. IHI '12, Association for Computing Machinery, New York, NY, USA (Jan 2012), https://dl.acm.org/doi/10.1145/2110363.2110464
31. Wang, L., Ma, C., Feng, X., et al.: A survey on large language model based autonomous agents. Frontiers of Computer Science **18**(6), 186345 (Mar 2024), https://doi.org/10.1007/s11704-024-40231-1
32. Wang, Y., Zhong, W., Huang, Y., et al.: Agents in Software Engineering: Survey, Landscape, and Vision (Sep 2024), http://arxiv.org/abs/2409.09030
33. Yetiştiren, B., Özsoy, I., Ayerdem, M., Tüzün, E.: Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT (Oct 2023), http://arxiv.org/abs/2304.10778